

Conecta em memória

Dez desafios, e cada um é um pedaço de verdade do sistema que você está construindo. Você escreve o método, o programa testa sozinho, e a mini-tela ao lado roda o SEU código com os dados que você digitar.

a conta

a lista

entrar

já existe

o porteiro

buscar conta

numerar

sugerir login

trocar senha

remover

10 abas · uma aula de 4h · 3 assuntos novos: List, foreach e while

Os dez desafios

CADA UM E UM PEDACO DO CONECTA

#	Metodo	O que e	Tipo
1	CriarUsuario	o molde e o objeto	revisao
2	CriarConta	guardar varias contas	novo
3	Autenticar	achou, ou nao achou	novo
4	LoginExiste	o login e unico	revisao
5	ValidarCadastro	quatro if em ordem	revisao
6	BuscarPorId	o quarto metodo do DAO	revisao
7	Numerar	quando a posicao importa	revisao
8	SugerirLogin	sem saber quantas vezes	novo
9	TrocarSenha	o objeto na lista e o mesmo objeto	revisao
10	Remover	mexer na lista enquanto percorre	revisao

FICA DE FORA



Nao entra hoje

- banco de dados, MySQL e SQL
- { get; set; } - as propriedades
- construtor
- heranca, polimorfismo e encapsulamento
- try / catch
- a classe Postagem e o mural



Quatro deles sao o seu DAO

Os desafios 2, 3, 4 e 6 sao, letra por letra, os quatro metodos que voce escreve hoje a noite na Parte 7 da sua apostila. Nao e exercicio de faz de conta.

LEIA ISTO ANTES DE COMECAR

Como o aplicativo funciona

Cada assunto ocupa duas paginas: a primeira explica e mostra um exemplo, a segunda traz o desafio que voce escreve no computador. Leia a primeira, faca a segunda.

O CICLO, E ELE NAO TEM ATALHO

	o que fazer
1	Leia a aba e escreva o metodo em Desafios.cs
2	Salve com Ctrl + S
3	FECHE o programa
4	Rode de novo com F5



O passo 3 nao e opcional

O seu codigo e compilado quando voce roda. Com o programa aberto, editar o arquivo nao muda nada na tela - e o botao Conferir de novo tambem nao.

AS DUAS COISAS QUE ELE FAZ



Testa

Compara o que o seu metodo devolveu com o esperado, teste a teste. Passou em todos? A aba fica verde e a proxima destrava.



Roda de verdade

Na mini-tela de cada aba, com os dados que VOCE digitar. Ela so liga depois que os testes passam - primeiro o codigo certo, depois a tela.

AS CORES DO CODIGO



linha com barra roxa = voce
escreve agora

// ...

cinza e em italico = ja
esta no arquivo, nao
redigite

A memoria do programa

As contas que voce criar ficam numa lista dentro do programa. E a MESMA lista nas dez abas: a conta que voce cadastrar na aba 1 e a conta com que voce entra na aba 3.

Fechou o programa, a lista esvazia. Abriu de novo, ela nasce vazia outra vez.



Isso NAO e defeito

E a razao de existir banco de dados. Enquanto o programa esta aberto, o dado mora na memoria; quando ele fecha, a memoria vai junto. Guardar de verdade e o que a Parte 11 da sua apostila faz - e por isso ela existe.



O botao Semear

Toda mini-tela tem um. Ele poe duas contas de mentira - ana / 1234 e bruno / abcd - para voce ter com o que trabalhar sem precisar cadastrar tudo de novo a cada F5.

O QUE VOCE VAI VER NA TELA

#1	Ana Souza	ana	1234
#2	Bruno Lima	bruno	abcd
#3	Carla Dias	carla	9999

O numero com # e o Id da conta. A partir do desafio 2, quem calcula esse numero e o SEU codigo - se ele aparecer como #0, voce ja sabe o que faltou.



Se o seu laco nao terminar

O programa espera 2 segundos, desiste e avisa. Ele nao congela, e os outros desafios continuam funcionando normalmente.

PARTE

1

Guardar as contas

- A conta - o molde e o objeto
- A lista - guardar varias contas

1

O MOLDE E O OBJETO

A conta

A classe é o molde. Ela diz o que toda conta do Conecta guarda: um Id, um Nome, um Login e uma Senha. Enquanto é só o molde, não existe conta nenhuma - existe a receita.

O objeto é a conta de verdade, e ele nasce com new. Cada new produz um objeto separado, com os próprios valores: duas contas feitas a partir do mesmo molde não dividem nada.

Escrever `u.Nome = nome` quer dizer: no objeto `u`, guarde no campo `Nome` o valor que chegou no parametro `nome`. São duas coisas diferentes com nomes parecidos - o de fora e o valor que chegou, o de dentro e onde ele fica guardado.

No fim, o método entrega o objeto com `return`. Quem chamou fica com a conta na mão.



No seu projeto

Esta é a classe `Usuario` da Parte 6 da sua apostila. Hoje a noite ela ganha o Id e vira o coração do `UsuarioDAO`.

EXEMPLO

```
Usuario u = new Usuario();  
  
u.Nome = "Ana Souza";  
u.Login = "ana";  
  
return u;
```

O new cria. As atribuições preenchem. O return entrega.



Guarde esta frase

A classe é o molde. O objeto nasce com new. O `return` entrega o objeto para quem pediu.

1

VOCE ESCREVE ESTA PARTE

Desafio 1: CriarUsuario

Crie um objeto Usuario com new, guarde nele o nome, o login e a senha que chegaram, e devolva o objeto. O Id nao e problema seu: quem da o numero e o desafio 2.

O METODO QUE VOCE COMPLETA

```
public static Usuario CriarUsuario(string nome, string login,
                                   string senha)
{
    // TODO
}
```



A dica

Sao cinco linhas: uma com new, tres atribuicoes e o return. Os tres valores ja chegaram prontos entre parenteses - nao invente texto entre aspas.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

Ana Souza / ana / 1234 -> devolve a conta com os tres campos preenchidos

Bruno Lima / bruno / abcd -> o nome muda junto (nada fixo entre aspas)

senha em branco -> o metodo guarda o que chegou e nao julga o dado



o nome fixo entre aspas

Escrever u.Nome = "Ana"; passa no primeiro teste e falha em todos os outros: toda conta criada sai chamada Ana. O valor tem que vir do parametro, sem aspas.

2

GUARDAR VARIAS CONTAS

A lista



Assunto NOVO desta aula

Este assunto e novo. Hoje sao tres: a List, o foreach e o while. Os outros sete desafios sao revisao aplicada ao Conecta. Aprenda com calma, sem correr.

Uma variavel guarda um valor so. Guarde a conta da Maria numa variavel e depois a do Bruno na mesma variavel: a Maria nao fica ao lado dele, ela e substituida por ele.

A List resolve isso. Ela guarda uma fila de objetos e cresce sozinha, sem voce dizer de antemao quantos serao. `List<Usuario>` se le lista DE Usuario: o que esta entre os sinais de maior e menor e o tipo do que ela guarda dentro.

Sao duas operacoes hoje. `contas.Add(u)` poe u no fim da fila. `contas.Count` diz quantas ja estao la dentro - e repare que `Count` nao tem parenteses.

O `Id` serve para diferenciar uma conta da outra. Se ja existem duas, a proxima e a numero 3: e o `Count` mais um. Sem o mais um, a primeira conta nasceria com `Id 0`.

EXEMPLO

```
List<Usuario> contas = new List<Usuario>();

contas.Add(ana);           // agora tem 1
contas.Add(bruno);         // agora tem 2

int quantas = contas.Count; // 2
```

A lista comeca vazia e cresce a cada Add. O Count sempre diz o tamanho de agora.



Guarde esta frase

A List guarda varios e cresce sozinha. `Add` acrescenta, `Count` conta. O primeiro `Id` e `Count + 1`.

No seu projeto: E o `UsuarioDAO.CriarConta` da Parte 7. La a lista se chama `_usuarios` e mora dentro da classe; aqui ela chega pela porta, como parametro. O miolo e o mesmo.

2

VOCE ESCREVE ESTA PARTE

Desafio 2: CriarConta

De ao usuario um Id igual a quantidade de contas mais um, acrescente ele na lista, e devolva o Id. A primeira conta tem que sair com Id 1, nao 0.

O METODO QUE VOCE COMPLETA

```
public static int CriarConta(List<Usuario> contas, Usuario u)
{
    // TODO
}
```



A dica

O Count diz quantas ja estao la dentro. Some 1 antes de guardar no Id. E acrescentar de fato na lista e um passo separado do return - os testes conferem o tamanho da lista depois.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

lista vazia -> a primeira conta nasce com Id 1, e a lista fica com 1

ja havia 1 conta -> a nova recebe Id 2

ja havia 3 contas -> a nova recebe Id 4

e a lista precisa CRESCER: calcular o Id sem o Add nao vale



calculou o Id e nao guardou

Dar o numero ao objeto e acrescentar ele na lista sao dois passos separados. Quem faz so o primeiro devolve o Id certo, ve o numero aparecer na tela, e a lista continua exatamente do mesmo tamanho.

P A R T E

2

Entrar e conferir

- Entrar - achou, ou não achou
- Já existe - o login é único
- O porteiro - quatro if em ordem

3

ACHOU, OU NAO ACHOU

Entrar



Assunto NOVO desta aula

Este assunto e novo. Hoje sao tres: a List, o foreach e o while. Os outros sete desafios sao revisao aplicada ao Conecta. Aprenda com calma, sem correr.

Para saber se alguem pode entrar, o sistema olha as contas uma por uma. O foreach faz exatamente isso: `foreach (Usuario u in contas)` se le para cada Usuario, que eu chamo de u, dentro de contas.

O foreach nao pede indice, nao pede tamanho e nao erra a conta: ele passa por todos, do primeiro ao ultimo, e para sozinho.

Entrar exige as duas coisas ao mesmo tempo: o login tem que bater E a senha tem que bater. Isso e o &&. Com || bastaria uma das duas, e qualquer pessoa entraria na conta de qualquer outra sabendo so o login.

E quando ninguem bate? O metodo devolve null, que quer dizer procurei e nao achei. O lugar de cada return muda tudo: um vai DENTRO do laco, o outro DEPOIS dele - e so ali voce tem certeza de que olhou todas.

EXEMPLO

```
foreach (Usuario u in contas)
{
    if (u.Id == procurado)
    {
        return u;           // achou: sai na hora
    }
}

return null;               // olhou todas e nao achou
```

Dois return em lugares diferentes. Este exemplo procura pelo Id; o seu desafio procura por login e senha ao mesmo tempo.



Guarde esta frase

O foreach passa por todos. O return de achou sai de dentro; o null so sai depois do laco.

3

VOCE ESCREVE ESTA PARTE

Desafio 3: Autenticar

Percorra a lista com foreach e devolva a conta em que o login E a senha batem com os que chegaram. Se o foreach terminar sem achar nenhuma, devolva null. Sao dois return, em lugares diferentes.

O METODO QUE VOCE COMPLETA

```
public static Usuario Autenticar(List<Usuario> contas,  
                                string login, string senha)  
{  
    // TODO  
}
```



A dica

O null so pode aparecer DEPOIS que o foreach terminar. Dentro do laco cabe so o caso de achar: compare os dois campos com == ligados por &&, e se bater, devolva aquela conta na hora.

No seu projeto: E o UsuarioDAO.Autenticar do Passo 42: ele devolve a conta, e nao um sim ou nao, porque o sistema precisa do Id de quem entrou.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

ana / 1234 -> devolve a conta da Ana

bruno / abcd -> devolve a SEGUNDA conta (aqui o null dentro do laco quebra)

login certo com a senha do outro -> nao entra (aqui o || no lugar do && quebra)

quem nao existe nao entra

lista vazia -> o foreach nao entra nenhuma vez



o null dentro do laco

Se o return null ficar dentro do foreach, o metodo desiste na primeira conta que nao bate. Funciona para quem se cadastrou primeiro e falha para todo mundo depois - e quem escreveu jura que testou.

ONDE CADA UM FICA, E POR QUE

A pagina dos dois return

Procurar alguma coisa numa lista tem sempre a mesma forma, e ela aparece em CINCO desafios de hoje: o 3, o 4, o 6, o 9 e o 10. Vale a pena entender uma vez so.

CERTO

```
foreach (Usuario u in contas)
{
    if (u.Login == login)
    {
        return u;           // achou: sai na hora
    }
}

return null;               // olhou TODAS e nao achou
```



A frase que resolve

Achou sai de dentro. Nao achei so sai depois.

ERRADO

```
foreach (Usuario u in contas)
{
    if (u.Login == login)
    {
        return u;
    }
    return null;           // <- AQUI
}
```

Com o null la dentro, o metodo desiste na PRIMEIRA conta que nao bate. Ele nunca chega na segunda.



Por que ninguem percebe

Voce testa com a sua propria conta, que foi a primeira que voce cadastrou. Funciona. So quebra para quem se cadastrou depois - e ai voce jura que testou.

4

O LOGIN É ÚNICO

Já existe

Se duas contas tiverem o mesmo login, o Autenticar da aba anterior vai achar sempre a primeira - e a segunda pessoa nunca entra na própria conta. Não é detalhe: é o sistema mentindo para o usuário.

A defesa é conferir antes de gravar. O método percorre as contas e responde uma pergunta de sim ou não: este login já está em uso?

O desenho é o mesmo do Autenticar, com uma diferença: em vez de devolver a conta, devolve true. E em vez de null no fim, devolve false.

O true sai de dentro do laço assim que achar. O false só pode sair depois que o laço acabar, porque só aí você sabe que olhou todas.



No seu projeto

É a quinta validação do btnCadastrar, no Passo 41: se o LoginExiste devolver true, o cadastro mostra a mensagem e para com return, sem criar a conta.

EXEMPLO

```
foreach (Usuario u in contas)
{
    if (u.Senha == senha)
    {
        return true;    // achou alguém com essa
        senha
    }
}

return false;          // olhou todas, ninguém usa
```

Mesmo desenho do desafio anterior. Muda o que sai: true e false no lugar da conta e do null.



Guarde esta frase

Login repetido faz o Autenticar achar sempre a primeira conta. Confira ANTES de gravar.

4

VOCE ESCREVE ESTA PARTE

Desafio 4: LoginExiste

Percorra as contas e devolva verdadeiro se o login ja estiver em uso. Se o foreach terminar sem achar nenhum igual, devolva falso.

O METODO QUE VOCE COMPLETA

```
public static bool LoginExiste(List<Usuario> contas, string login)
{
    // TODO
}
```



A dica

O true sai de dentro do laço assim que achar. O false so pode sair depois que o laço acabar, porque so ai voce sabe que olhou todas.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

ana ja existe -> verdadeiro

bruno tambem, e ele e o SEGUNDO (aqui o false dentro do laço quebra)

carla nao existe -> falso, e so depois de olhar as duas

lista vazia -> nao existe



o false dentro do laço

Com o return false dentro do foreach, o metodo responde nao existe depois de olhar so a primeira conta. Cadastre duas pessoas e tente repetir o login da segunda: passa direto.

O UsuarioDAO

Quatro desafios de hoje sao, letra por letra, os quatro metodos do UsuarioDAO da Parte 7 da sua apostila. Muda uma coisa so, e vale entender qual.

AQUI, NO SIMULADOR

```
public static bool LoginExiste(List<Usuario> contas,
                               string login)
{
    foreach (Usuario u in contas) { ... }
}
```

LA, NO SEU CONECTA

```
private static List<Usuario> _usuarios
    = new List<Usuario>();

public static bool LoginExiste(string login)
{
    foreach (Usuario u in _usuarios) { ... }
}
```

O traco baixo do _usuarios e convencao para campo privado: serve para voce bater o olho e saber que aquilo e de dentro da classe.

OS QUATRO METODOS

Desafio	Metodo do DAO
2	CriarConta
3	Autenticar
4	LoginExiste
6	BuscarPorId



O corpo e identico

Quando UM metodo precisa da lista, ela entra pela porta, como parametro. Quando TODOS os metodos da classe precisam da mesma lista, voce para de passar e guarda dentro de casa. Foi por isso que o UsuarioDAO nasceu.

5

QUATRO IF EM ORDEM

O porteiro

Antes de criar a conta, o cadastro confere o que foi digitado. Cada regra e um if; se a regra for violada, o metodo devolve a mensagem e acaba ali mesmo.

E por isso que se chama porteiro: ele barra na porta. O return nao serve so para entregar um valor - ele tambem interrompe o metodo na hora, e nada depois dele roda.

A ORDEM importa. Quem escreve a regra da senha antes da regra do nome faz o sistema reclamar da senha para quem esqueceu de digitar o nome.

O || aparece quando duas situacoes diferentes dao o mesmo erro: senha curta demais OU comprida demais e o mesmo problema, com a mesma mensagem.

EXEMPLO

```
if (email == "")
{
    return "Informe o email.";
}

if (idade < 16 || idade > 120)
{
    return "Idade invalida.";
}

return ""; // passou por tudo: nao ha erro
```

Texto vazio quer dizer nao achei problema nenhum. O ultimo return fica FORA de todos os if.



No seu projeto

Sao as quatro validacoes da Parte 5. La cada uma mostra um MessageBox e para com return; aqui elas devolvem a mensagem, para o corretor poder conferir.



Guarde esta frase

Cada regra e um if com o seu return. A ordem das regras e a ordem das mensagens.

5

VOCE ESCREVE ESTA PARTE

Desafio 5: ValidarCadastro

Devolva a mensagem do PRIMEIRO problema, nesta ordem: nome vazio, login vazio, senha fora de 4 a 12 caracteres, senha diferente da confirmacao. As mensagens exatas estao nos comentarios do metodo. Se estiver tudo certo, devolva texto vazio.

O METODO QUE VOCE COMPLETA

```
public static string ValidarCadastro(  
    string nome, string login, string senha, string confirmar)  
{  
    // TODO  
}
```



A dica

Sao quatro if, cada um com o seu return, na ordem do enunciado. Use `senha.Length` para medir, e o `||` para juntar curta demais e comprida demais na mesma regra. O return do texto vazio fica fora de todos os if.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

nome vazio -> reclama do nome, e e a primeira regra

login vazio -> reclama do usuario

senha com 2 caracteres -> curta demais

senha com 13 -> comprida demais, mesma mensagem (aqui entra o `||`)

confirmacao diferente -> ultima regra

tudo certo -> texto vazio

if sem return: Sem o return dentro do if, o metodo confere a regra, nao faz nada com o resultado e segue em frente. O cadastro aceita tudo, e quem escreveu jura que a validacao esta la - e esta mesmo, so nao serve para nada.

P A R T E

3

Percorrer de outros jeitos

- Buscar conta - o quarto metodo do DAO
- Numerar - quando a posicao importa
- Sugerir login - sem saber quantas vezes

6

O QUARTO METODO DO DAO

Buscar conta

Depois que alguém entra, o sistema guarda o Id de quem esta logado - nao a pessoa inteira. Quando precisa do nome de novo, busca pelo Id.

Na Atividade 2 e assim que o mural mostra quem escreveu cada recado: a postagem guarda o UsuarioId, e o feed busca a conta por esse numero.

O metodo e irmao do Autenticar: mesmo foreach, mesmo par de return em lugares diferentes. Muda so a pergunta do if, que agora compara numero em vez de texto.

Com este metodo pronto o seu UsuarioDAO fica completo: CriarConta, Autenticar, LoginExiste e BuscarPorId. Sao os quatro que voce escreve hoje a noite, no projeto de verdade.



No seu projeto

E o UsuarioDAO.BuscarPorId do Passo 44. Na Parte 11 ele vira um SELECT ... WHERE Id = @id, e nenhuma tela precisa mudar.

EXEMPLO

```
// procurando pelo login, e nao pelo Id:

foreach (Usuario u in contas)
{
    if (u.Login == procurado)
    {
        return u;
    }
}

return null;
```

O mesmo desenho de sempre. No seu desafio a comparacao e com numero, e numero se compara com == igual a texto.



Guarde esta frase

Buscar por Id fecha o UsuarioDAO: sao quatro metodos, e voce ja escreveu todos.

6

VOCE ESCREVE ESTA PARTE

Desafio 6: BuscarPorId

Percorra as contas e devolva aquela cujo Id for igual ao pedido. Se nenhuma tiver esse Id, devolva null.

O METODO QUE VOCE COMPLETA

```
public static Usuario BuscarPorId(List<Usuario> contas, int id)
{
    // TODO
}
```



A dica

E o mesmo desenho do desafio 3: foreach, if, return dentro, null depois. Muda so a comparacao - agora e `u.Id == id`, com numero.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

Id 1 -> a primeira conta

Id 2 -> a segunda (aqui o null dentro do laco quebra)

Id que nao existe -> null

lista vazia -> null



usar o resultado sem conferir

Quem chama `BuscarPorId` e usa `.Nome` direto leva `NullReferenceException` no dia em que o Id nao existir. Devolver null nao e erro - e resposta. Quem chamou e que precisa perguntar antes de usar.

7

QUANDO A POSICAO IMPORTA

Numerar

O foreach e ótimo para passar por todos, mas ele não sabe dizer em que posição você está. Quando o número da linha faz parte do resultado, ele não serve.

O for sabe, porque ele carrega o contador. As três partes entre parênteses são: onde começa, até quando continua, e quanto anda a cada volta.

Para pegar a conta de uma posição use `contas[i]`, com colchetes, e não com ponto. A posição 0 é a primeira da lista, e a última é `Count` menos um.

Guarde o critério, porque ele vale para o resto do curso: preciso saber em que posição esse item está? Se sim, for. Se não, foreach.

EXEMPLO

```
string texto = "";

for (int i = 0; i < notas.Count; i++)
{
    texto = texto + (i + 1) + ") " + notas[i] + ";
}

return texto;    //  "1) 7,5; 2) 9; "
```

O i começa em 0, mas na tela a primeira linha é o número 1. Por isso o i + 1.



Guarde esta frase

Preciso da posição? for. Não preciso? foreach. A posição 0 é o número 1 na tela.

7

VOCE ESCREVE ESTA PARTE

Desafio 7: Numerar

Monte um texto com as contas numeradas, na ordem da lista: [Ana Souza, Bruno Lima] vira "1. Ana Souza | 2. Bruno Lima | ". Repare no ponto, na barra e nos espacos - inclusive depois do ultimo nome. Lista vazia devolve texto vazio.

O METODO QUE VOCE COMPLETA

```
public static string Numerar(List<Usuario> contas)
{
    // TODO
}
```



A dica

Comece com um texto vazio e va grudando um pedaco por volta. O numero que aparece na tela e $i + 1$; a conta daquela volta e `contas[i]`. Nao esqueca o espaco depois da barra, inclusive no ultimo.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

duas contas -> numeradas 1 e 2, com barra e espaco no fim

uma conta -> começa em 1, e nao em 0

tres contas -> a numeracao acompanha ate o fim

lista vazia -> texto vazio, e o proprio for resolve isso sozinho



esquecer o mais um

A lista começa na posicao 0, mas ninguem le uma tela que começa em 0. Sem o $i + 1$ a primeira conta aparece numerada como 0, e a ultima fica com o numero da penultima.

8

SEM SABER QUANTAS VEZES

Sugerir login



Assunto NOVO desta aula

Este assunto e novo. Hoje sao tres: a List, o foreach e o while. Os outros sete desafios sao revisao aplicada ao Conecta. Aprenda com calma, sem correr.

O login desejado esta ocupado. O sistema podia so recusar, mas todo site bom faz melhor: oferece um parecido que esteja livre. E o que acontece quando voce cria um email e ele sugere o seu nome com um numero no fim.

Aqui o for nao serve, e o motivo e simples: ninguem sabe quantas tentativas serao necessarias. Pode ser uma, pode ser trinta. O while existe exatamente para isso - ele repete enquanto a condicao for verdadeira, sem numero combinado.

O while tem as mesmas tres pecas do for, so que espalhadas: o valor inicial fica ANTES do laco, o teste fica entre parenteses, e o avanco fica DENTRO do corpo.

O avanco dentro do corpo e a peca que todo mundo esquece. Sem ele a condicao nunca muda, o laco nunca termina, e o programa trava de verdade - sem mensagem de erro nenhuma.

EXEMPLO

```
int valor = 1;

while (valor < 1000)
{
    valor = valor * 2;    // 1, 2, 4, 8, 16 ...
}

return valor;    // 1024
```

*Ninguem calculou quantas voltas serao. O laco para quando a condicao deixa de ser verdadeira. Tire o valor * 2 e o programa congela.*



Guarde esta frase

Sei quantas voltas? for. Nao sei? while - e o passo fica DENTRO do corpo, ou trava.

8

VOCE ESCREVE ESTA PARTE

Desafio 8: SugerirLogin

Se o login desejado estiver livre, devolva ele mesmo. Se estiver ocupado, junte um numero no fim, começando em 2, ate achar um livre: ana ocupado vira ana2; se ana2 tambem estiver ocupado, vira ana3. Use while, e pergunte se esta ocupado com `Ajuda.LoginEmUso(contas, tentativa)`.

O METODO QUE VOCE COMPLETA

```
public static string SugerirLogin(List<Usuario> contas,
                                string desejado)
{
    // TODO
}
```



A dica

Antes do laço: uma variavel com a tentativa, começando igual ao desejado, e um numero começando em 1. Enquanto a tentativa estiver em uso, aumente o numero e monte a tentativa de novo juntando o desejado com o numero. Devolva a tentativa depois do laço.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

login livre -> devolve ele mesmo, sem entrar no laço

ana ocupado -> ana2

ana e ana2 ocupados -> ana3 (duas voltas)

tres ocupados seguidos -> ana4

lista vazia -> nada esta ocupado



o while sem o passo

Se a tentativa nao mudar dentro do laço, a condicao continua verdadeira para sempre e a janela congela. Nao aparece erro nenhum: o programa simplesmente para de responder, e voce precisa fecha-lo pelo Gerenciador de Tarefas.

O laço que nao termina

O for carrega as tres pecas entre parenteses, na mesma linha, e por isso e dificil esquecer alguma. O while espalha as mesmas tres - e a terceira fica longe das outras duas, dentro do corpo.

```
int i = 5;           // 1  ANTES do laço

while (i >= 1)       // 2  entre parenteses
{
    Console.WriteLine(i);
    i--;             // 3  DENTRO do corpo
}
```



Tire a linha 3 e veja

Sem o i--, o i continua valendo 5 para sempre. A condicao i >= 1 nunca fica falsa, e o laço nunca termina.

O QUE ACONTECE DE VERDADE

Nao aparece mensagem de erro nenhuma. O programa simplesmente para de responder: a janela congela, o mouse nao clica em nada, e nada na tela muda.



Aqui ele NAO congela

O programa roda o seu código com prazo de 2 segundos. Passou disso, ele desiste e avisa - e os outros desafios continuam funcionando. Fora daqui, no seu projeto, só o Gerenciador de Tarefas resolve.



Sei quantas voltas? for.

Nao sei? while. Contar de 1 a 10 e for. Tentar ate achar um login livre e while, porque ninguem sabe quantas tentativas serao.

PARTE

4

Mudar e tirar

- Trocar senha - o objeto na lista e o mesmo objeto
- Remover - mexer na lista enquanto percorre

9

O OBJETO NA LISTA E O MESMO OBJETO

Trocar senha

Trocar a senha é pedido comum de qualquer sistema. Antes de trocar, é preciso provar que a pessoa é dona da conta - por isso o método recebe a senha atual, e não só o login.

A busca é a de sempre: um foreach, um if com os dois campos ligados por &&, e dois return em lugares diferentes.

O que muda é o que você faz depois de achar. Aqui você não devolve a conta: você **MUDA** ela e devolve verdadeiro.

E aqui está o ponto da aba: o u que o foreach entrega **NAO** é uma cópia. É o mesmo objeto que está guardado na lista. Escrever `u.Senha = nova` muda a conta lá dentro, e não existe passo de guardar de volta.



No seu projeto

Trocar a senha não está na Atividade 1, mas o raciocínio está: e assim que qualquer edição funciona. Na Atividade 2 é o que acontece quando alguém edita o próprio recado.

EXEMPLO

```
// o objeto da lista é o objeto da variável:  
  
Usuario primeiro = contas[0];  
primeiro.Nome = "Outro nome";  
  
// contas[0].Nome também virou "Outro nome".  
// Não existe devolver para a lista.
```

Uma caixa só, com dois nomes apontando para ela. Mexeu por um nome, mudou para os dois.



Guarde esta frase

O objeto que o foreach entrega é o mesmo que está na lista. Mudou nele, mudou lá.

9

VOCE ESCREVE ESTA PARTE

Desafio 9: TrocarSenha

Ache a conta em que o login E a senha atual batem, troque a senha dela pela nova, e devolva verdadeiro. Se nao achar, devolva falso e nao mude nada. Nao precisa tirar da lista e por de volta.

O METODO QUE VOCE COMPLETA

```
public static bool TrocarSenha(List<Usuario> contas, string login,
                                string atual, string nova)
{
    // TODO
}
```



A dica

E o mesmo desenho do desafio 3: foreach, if com == e && nos dois campos. Achou? atribua a senha nova naquele objeto e devolva true na hora. O false so sai depois do laco.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

ana com a senha certa -> troca, e a conta fica com a senha nova

bruno e a SEGUNDA conta (aqui o false dentro do laco quebra)

senha atual errada -> nao troca, e a senha continua a antiga

login que nao existe -> falso

lista vazia -> o foreach nao entra nenhuma vez



trocar sem conferir a senha

Quem procura so pelo login troca a senha de qualquer pessoa sabendo o nome de usuario dela. A senha atual nao e formalidade: e a prova de que quem pediu e o dono.

10

MEXER NA LISTA ENQUANTO PERCORRE

Remover

Excluir a conta e o ultimo pedido que todo sistema precisa atender. A busca e a mesma de sempre; o que muda e que agora voce nao le a lista - voce MEXE nela.

Para tirar um item existe `contas.Remove(u)`, que procura aquele objeto e o retira. A lista encolhe na hora, e quem estava depois anda uma posicao para tras.

E ai mora a armadilha do dia: o `foreach` nao aceita que a lista mude enquanto ele a percorre. Se voce remover e continuar o laço, o programa estoura com `InvalidOperationException - Collection was modified`.

A saida e simples, e voce ja usou nove vezes hoje: saia do laço na MESMA hora em que remover. O `return` faz as duas coisas de uma vez - devolve a resposta e encerra o metodo.

EXEMPLO

```
// ERRADO - o laço continua depois de mexer na
// lista:

foreach (Usuario u in contas)
{
    if (u.Id == id)
    {
        contas.Remove(u);    // a lista mudou...
    }
}                             // ...e aqui o foreach
// estoura
```

Este exemplo mostra o que NAO fazer. Faltava uma linha para consertar - e ela e a mesma que voce usou nos desafios 3, 4 e 6.



No seu projeto

Na Atividade 2 e o botao Apagar do mural: so o dono pode apagar o proprio recado. O DAO faz um `DELETE ... WHERE Id = @id`, e o feed recarrega a lista.



Guarde esta frase

Removeu dentro do `foreach`? Saia do laço na mesma hora, ou o programa estoura.

10

VOCE ESCREVE ESTA PARTE

Desafio 10: Remover

Ache a conta com aquele Id, tire ela da lista com contas. Remove e devolva verdadeiro. Se nenhuma tiver esse Id, devolva falso e nao mexa na lista.

O METODO QUE VOCE COMPLETA

```
public static bool Remover(List<Usuario> contas, int id)
{
    // TODO
}
```



A dica

Ache como sempre: foreach e if. Depois do Remove, saia do laco na mesma hora - o return true faz isso. O false so sai depois que o laco terminar.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

remove a do meio -> sobram Ana e Carla, nesta ordem

remove a primeira -> sobra Bruno

remove a ultima -> o laco chega ate o fim antes de achar

Id que nao existe -> falso, e a lista fica intacta

lista vazia -> falso



remover e continuar o laco

O foreach guarda por onde estava. Tirar um item da lista no meio do caminho quebra essa contagem, e o C# prefere estourar a te devolver dado errado. Nao ha erro de compilacao: o programa compila, roda, e para na hora.

A lista que muda no meio

O foreach guarda por onde ele estava. Se a lista mudar de tamanho no meio do caminho, essa marca deixa de valer - e o C# prefere parar o programa a te devolver dado errado.

```
foreach (Usuario u in contas)
{
    if (u.Id == id)
    {
        contas.Remove(u);    // a lista mudou...
    }
    // ...e aqui estoura
}
```

`InvalidOperationException: Collection was modified;
enumeration operation may not execute.`



Nao ha erro de compilacao

O programa compila, roda, e para na hora em que voce clica. Erro que so aparece rodando tem nome: erro em tempo de execucao.

O CONSERTO E UMA LINHA

```
contas.Remove(u);
return true;    // <- esta
```

O return faz duas coisas de uma vez: devolve a resposta e encerra o metodo. Encerrando o metodo, o foreach nunca chega a dar a volta seguinte - e nao tem como reclamar.



E o mesmo return de sempre

O de achou sai de dentro do laco. Voce ja escreveu isso nos desafios 3, 4 e 6. Aqui ele deixa de ser so a resposta e passa a ser o que salva o programa.

A folha de revisao

#	Assunto	Guarde esta frase
1	A conta	A classe e o molde. O objeto nasce com new. O return entrega o objeto para quem pediu.
2	A lista	A List guarda varios e cresce sozinha. Add acrescenta, Count conta. O primeiro Id e Count + 1.
3	Entrar	O foreach passa por todos. O return de achou sai de dentro; o null so sai depois do laco.
4	Ja existe	Login repetido faz o Autenticar achar sempre a primeira conta. Confira ANTES de gravar.
5	O porteiro	Cada regra e um if com o seu return. A ordem das regras e a ordem das mensagens.
6	Buscar conta	Buscar por Id fecha o UsuarioDAO: sao quatro metodos, e voce ja escreveu todos.
7	Numerar	Preciso da posicao? for. Nao preciso? foreach. A posicao 0 e o numero 1 na tela.
8	Sugerir login	Sei quantas voltas? for. Nao sei? while - e o passo fica DENTRO do corpo, ou trava.
9	Trocar senha	O objeto que o foreach entrega e o mesmo que esta na lista. Mudou nele, mudou la.
10	Remover	Removeu dentro do foreach? Saia do laco na mesma hora, ou o programa estoura.

O que voce leva de hoje

Que uma lista guarda varios e cresce sozinha, e que percorrer ela tem sempre a mesma forma: um foreach, um if, e dois return em lugares diferentes. Voce escreveu isso cinco vezes hoje, em cinco desafios que pareciam diferentes.

Que escolher o laco e uma decisao, e nao um habito. Preciso da posicao? for. Nao preciso? foreach. Nao sei quantas voltas? while. Os tres fazem a mesma coisa; um deles faz sem te atrapalhar.

E que o dado que voce guardou some quando o programa fecha. Isso nao e defeito nem descuido: e exatamente o problema que o banco de dados existe para resolver, e e o que voce vai fazer na proxima aula.